

AN ALTERNATIVE COMBINING ARCHITECTURE FOR LARGE-SCALE SHARED-MEMORY MULTIPROCESSORS

Nian-Feng Tzeng

The Center for Advanced Computer Studies
University of Southwestern Louisiana
Lafayette, LA 70504
U. S. A.

ABSTRACT

This paper investigates an alternative combining architecture for removing detrimental impacts on the multiprocessor interconnection network due to *hot-spot contention*, which results in drastically degraded performance. Based on a *combining tree*, the proposed combining architecture has much lower hardware complexity than the prior combining network and yet can effectively eliminate hot-spot contention. Simulation results confirm that the mean latency of hot-spot requests through the alternative combining architecture is low, provided that the concurrent hot-spot number is reasonably small (say, no more than 8). A fault-tolerant technique for the combining architecture is also addressed to significantly enhance its reliability at a little extra cost. Since it is inexpensive and can be made fault-tolerant easily, this combining architecture appears readily suited for large-scale shared-memory multiprocessors.

1. INTRODUCTION

The multistage interconnection network (MIN) is popularly employed to connect processors and memory modules in a large-scale multiprocessor system. A regular MIN could experience drastic performance degradation if any memory module gets heavy access from a set of processors due to the synchronization, resource management, or other operations. A notorious phenomenon called *hot-spot contention* [1] has been discovered to be frequently existing in a shared-memory multiprocessor interconnected by the MIN. It is caused by a huge number of concurrent requests addressing a shared variable and can lead to a *saturation tree* developed across a MIN. Once developed, a saturation tree blocks not only the requests heading for the hot memory module but also all other requests whose paths share common links with the tree. It has been shown by Pfister and Norton [1] that the performance degradation due to even light hot-spot traffic can be remarkable.

Incorporating the combining capability in switching elements makes it possible to combine requests destined for the same location into a single request. A combining network is a regular MIN with every switching element equipped with the combining capability; it was proven capable of eliminating tree saturation effectively [1]. In this paper, we propose an alternative combining architecture that has lower hardware complexity than the combining network and yet can get rid of tree saturation in many situations. At the heart of this architecture is a combining tree, which can remove the adverse impact due to tree saturation by

combining hot-spot requests. The architecture is used in conjunction with a regular MIN to support high bandwidth even in the presence of hot-spots, provided that the total number of concurrent hot-spot locations in the system is reasonably small (say, no more than 8, which could be the case for many applications). A fault-tolerant design for this architecture is also addressed to substantially enhance its reliability using a little extra hardware.

The proposed combining architecture is an inexpensive alternative to the prior combining network. While it cannot handle as many concurrent hot-spots as the prior combining network, this architecture could be powerful enough to cope with hot-spot traffic in typical applications. For the architecture to work successfully, it requires that hot-spot traffic be differentiated from regular traffic so that hot-spot requests can be sent to the combining architecture for combining. Although not all hot-spot references are for the purposes of synchronization and resource management, we believe that the most important and persistent hot-spot traffic is due to heavy access of shared locations for these purposes, and their references are often made by utilizing a set of specific primitive instructions (e.g., FETCH&OP's [2]). This type of hot-spot traffic is possible to be detected by examining (a part of) the instruction op-code or tag at run time and can be handled effectively by our design.

2. ALTERNATIVE COMBINING ARCHITECTURE

The proposed combining architecture consists of a combining tree and a demultiplexer tree, as depicted in Fig. 1. The combining tree has N input ports, each connected to a processor, and an output port through which combined requests are delivered to the demultiplexer tree and then forwarded to their proper destinations. This combining architecture is used exclusively for dealing with hot-spot traffic identified at run time. Regular references and indistinguishable hot-spot references go through a separate regular MIN to reach their destinations.

2.1. Combining Tree

Structurally, the combining tree is a complete binary tree with $n = \log_2 N$ levels [3], where N is the network size. The combining tree consists of *combining elements*, which involves one combining queue, one de-combining logic, and one wait buffer, as shown in Fig. 2. Compared with the building block for the combining network suggested for the NYU Ultracomputer and the IBM RP3, this element has merely one half the component count and could be faster

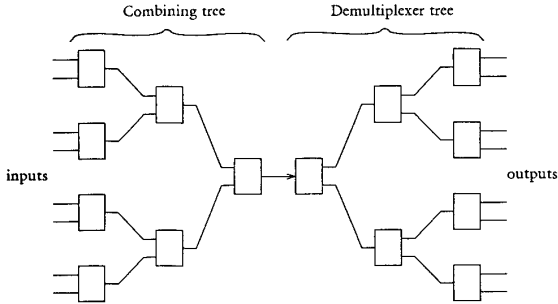


Fig. 1. An alternative combining architecture.

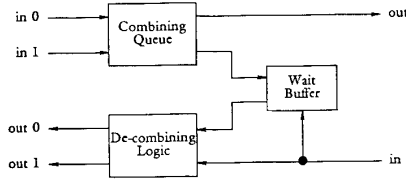


Fig. 2. The block diagram of combining elements.

because of its simplicity. The combining tree, being a tree structure, needs totally $(N-1)$ combining elements. In terms of the element count, the combining tree has only $O(1/\log_2 N)$ that of a combining network with the same size. More importantly, since the tree structure ensures that the replies of requests return to any node of the tree in the same order as their corresponding requests depart from the node, the wait buffer in the combining element is simply a first-in first-out (FIFO) queue. A FIFO queue is inexpensive and can be made sufficiently large to accommodate many combined requests.

Hot-spot requests destined for the same memory location are combined whenever they meet at a combining element on the way to the root. If a request is combined with another request at a combining element, only one representative request proceeds forwards, with a record which indicates this combining stored in the wait buffer of the element. This record enables replies for the two combined requests to be produced appropriately later on. On reaching the root, a request is forwarded in accordance with its address over the demultiplexer tree to its destination.

The distinct hot-spot locations existing in a system simultaneously are referred to as *concurrent hot-spots*. Suppose that the combining queue in a combining element is of length ξ . When the number of concurrent hot-spots is no larger than ξ , a newly arrived request can always find a combining partner or a free slot in the combining queue. For the same reason, an outgoing request is guaranteed to be able to leave a combining queue, keeping the traffic flow fluent. A typical application is likely to involve only a few concurrent hot-spots. As the number of hot-spots increases to a certain extent, they are no longer "hot" and even the regular MIN can then handle them. Thus, we are interested in the cases where the number of concurrent hot-spots in a system is limited to a low value (because only those cases would suffer from tree saturation seriously [4]).

Since the leaf nodes account for more than one half of the total tree nodes and the hot-spot rate is normally well

below 0.5, it is feasible to reduce the cost of the combining tree by replacing combining elements at the first level with multiplexers. If the hot-spot rate is low, elements at multiple front levels may be replaced without affecting its performance noticeably, giving rise to further cost reduction.

2.2. Demultiplexer Tree

A demultiplexer is much simpler than a combining element, so it is possible to construct the demultiplexer tree using components larger than 1×2 (e.g., 1×4 or 1×8), subject to the pin count limitation. The demultiplexer tree so constructed not only better matches the operation speed of the combining tree, but also lowers the latency of delivered requests. A demultiplexer tree for a sized N system consists of $O(N)$ components.

A memory module is assumed to be capable of accepting up to two requests during a network cycle, with one coming from the demultiplexer tree and the other from the regular MIN; the two requests can be satisfied simultaneously and independently. A combined request experiences a fixed latency when passing through the demultiplexer tree, as it never encounters a conflict. More importantly, a combined request leaving the combining tree earlier is guaranteed to return earlier, ensuring sequential consistency. This makes it sufficient to use a simple first-in first-out (FIFO) queue as the wait buffer in a combining element.

2.3. Simulation Results

To estimate the performance of our proposed combining architecture, we ran simulation under various concurrent hot-spot numbers (m) and request generation rates (g) for a system with size $N = 256$. Hot-spot rates (h) of 1% and 10% are considered. A hot-spot rate h means that issued requests have a probability of h being hot-spot ones, with the rest uniformly addressing all memory modules. We simulated the performance of hot-spot requests on the forward direction only, since that on the return direction is trivial.

Suppose that all hot-spot requests can be differentiated from regular requests and are sent to the combining architecture for combining. A combining element is assumed to have an unlimited combining capability, i.e., a hot-spot request can combine with a newly arrived one of the same kind, no matter whether it has already combined or not in the combining element. In practice, a hot-spot request stays in a combining element for only limited cycles, combined only with a small number of partners. The following assumptions are made in our simulation:

- (1). The combining tree and the demultiplexer tree operate at the same speed. The demultiplexer tree consists of 1×2 demultiplexers;
- (2). A combining queue of depth 8 (i.e., it can hold up to 8 requests) is incorporated in a combining element;
- (3). A request, on entering a combining queue, is compared simultaneously with all requests held in the queue;
- (4). A combining queue can accept up to 2 requests during a cycle. If a request heads for a full queue, it cannot proceed and remains in its original location;
- (5). Every hot-spot request has the same probability of addressing any one of the concurrent hot-spot locations, which are placed in memory modules equidistantly.

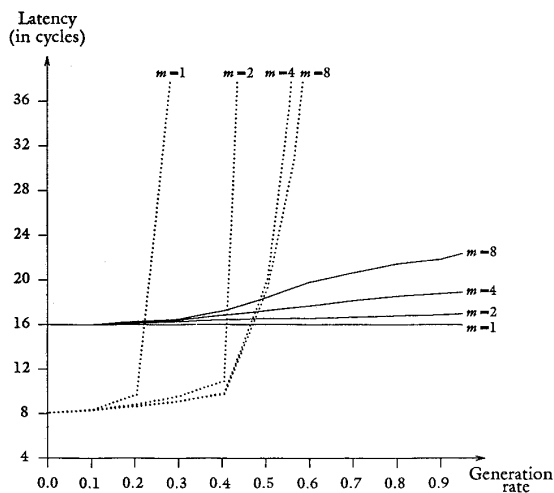


Fig. 3. Mean latency (L) versus generation rate (g) for various concurrent hot-spot numbers (m) under hot-spot rate $h = 1\%$. (The dotted lines denote the results of a regular MIN.)

The performance measure of interest is the mean latency of hot-spot requests, L (in terms of cycles). If an interconnection architecture can effectively handle hot-spot access, L is small; otherwise, L becomes unacceptably large. The latency of a request is counted from the time the request is generated until it leaves the interconnection.

In Fig. 3, mean latency (L) versus generation rate (g) is plotted for the concurrent hot-spot number ranging from 1 to 8 under the hot-spot rate of 1%. (Notice that the minimum possible latency in this case is 16 cycles, with 8 cycles over the combining tree and 8 cycles over the demultiplexer tree.) It can be seen that L stays low consistently and increases slightly as g or m grows, for $m \leq 8$. This indicates that the proposed architecture indeed can remove adverse hot-spot contention, as long as the concurrent hot-spot number is no more than 8. The mean latency of hot-spot requests through a regular MIN without employing this proposed combining architecture is included for comparison (assuming each switch output in the regular MIN has a buffer with capacity of 2 slots, and during a cycle, a switch output can receives up to 2 requests). As expected, the regular MIN soon saturates as the generation rate increases, despite its lower latency under extremely light traffic. (It should be noted that the mean latency in our architecture is reduced using a larger demultiplexer building block.)

Fig. 4 illustrates the same relationship under a high hot-spot rate (10%). Again, the mean latency remains reasonably low, suggesting that this combining architecture is capable of handling heavy hot-spot traffic. When compared with that in Fig. 3, L now is somewhat larger, and the gap grows as g or m increases. This is because a larger request generation rate or hot-spot number results in more hot-spot requests held at each combining queue, taking more time to traverse every level in the combining tree.

3. FAULT-TOLERANT DESIGN

The combining element is far more complex than a regular element or a demultiplexer, so it is critically important to incorporate fault-tolerance into the combining tree in

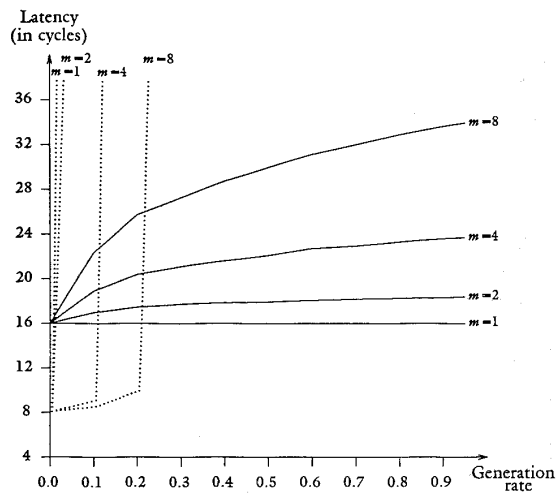


Fig. 4. Mean latency (L) versus generation rate (g) for various concurrent hot-spot numbers (m) under hot-spot rate $h = 10\%$. (The dotted lines denote the results of a regular MIN.)

order to meet the reliability requirement of the overall combining architecture. We propose a fault-tolerant design for our combining architecture as illustrated in Fig. 5(a). A 1×2 demultiplexer is added to each link in the combining tree so as to provide a bidirectional connection between the two upper (lower) incoming links of two neighboring combining elements at the same level. Two neighboring combining elements are *conjugate* with each other if their upper (and lower) incoming links are connected through the added demultiplexers. In Fig. 5(a), for example, the two combining elements at level 1 are conjugate.

The root node of the combining tree is duplicated to avoid becoming a single vulnerable point, with a demultiplexer added at the incoming link to each root node and appropriate connections established between the demultiplexers and the "two root nodes" as shown in Fig. 5(a). The root and its duplicate are also conjugate. The demultiplexer tree remains unchanged, except that a 2×2 switch is in place of a 1×2 demultiplexer at the root to accommodate two outputs from the "two roots" of the combining tree. The added demultiplexer is assumed very reliable (as it is much simpler than a combining element), so is the demultiplexer tree.

This fault-tolerant design is obtained by taking advantage of the fact that hot-spot requests can be combined at any combining element within the same level equally efficiently. After a combining element fails, all traffic to its two incoming links is enroute by the added demultiplexers and connections to its conjugate element, where the combining process takes place. As depicted in Fig. 5(b), hot-spot requests to the failed element in level 1 are sent to its conjugate element for combining. Although the request rate to the conjugate element is likely to grow due to enroute traffic, the performance of the combining tree would not degrade much after faults arise because combined requests which otherwise would proceed to the next level, might get further combined with newly arrived requests while being forced to stay in level 0 (as a result of contending for incoming links to the conjugate element).

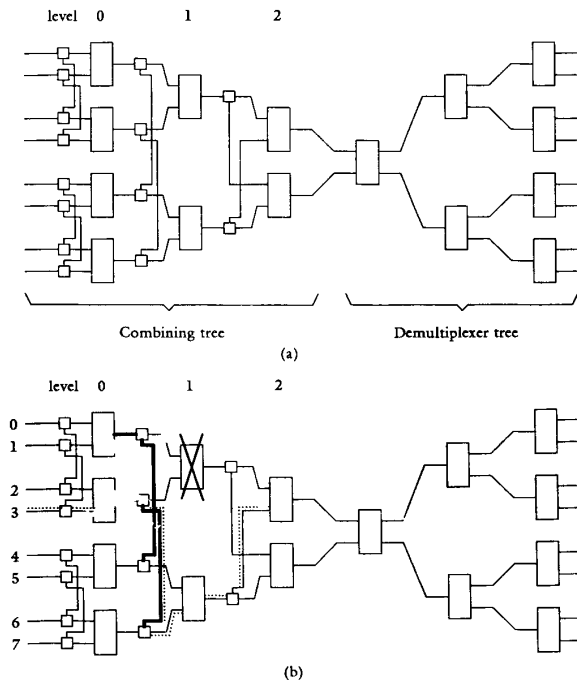


Fig. 5. (a) A fault-tolerant combining architecture. (b) Reconfiguration in response to a fault (the dotted lines indicate a path between primary input 3 and a root of the combining tree).

In order to ensure correct de-combining operations when replies are back, it is necessary that requests and replies take identical paths. This can be done by using an *indicator* for each request to keep track of the forward path traversed (in the combining tree). The indicator has the same length as a request's tag (i.e., which is used in the demultiplexer tree along the forward direction). Specifically, when a request proceeds in the combining tree, a value "1" (or "0") is assigned to the i^{th} bit of its indicator if it is enrounted (or not enrounted) at the added demultiplexer in level i . A hot-spot request entering from primary input 3 of Fig. 5(b), for example, generates an indicator with value "010" (because it is enrounted in level 1) when traveling along the forward direction.

This indicator is used together with the source address of the request (which is the tag of the reply) to find the correct path back to its origin, as follows: at level i of the combining tree along the return direction, the reply is directed to an output determined by the tag bit value, if the $(i-1)^{\text{th}}$ bit in its indicator is 0; otherwise, to an output determined by the complement of the tag bit value. The i^{th} bit of an indicator is also used for making the proper setting of an added demultiplexer in level i . Considering the return path of the above request originating from input 3. The tag for the return path is "011" (= 3) and the indicator is "010". No routing information is needed in passing through the demultiplexer tree (and at its root node, a request can be forwarded to an arbitrary output). At a combining element in level 2 (the first level in the combining tree on the return direction), the reply is sent over the lower output because the tag bit is "0" and the indicator bit examined is "1", for-

warding the reply to the lower element in level 1 and avoiding the failed element (see the path marked in Fig. 5(b)). At the element in level 1, the reply goes through the lower output because the tag bit is "1" and the indicator bit examined is "0". Since the value of indicator bit 1 is "1", the added demultiplexer is set in a way to direct the reply to the second combining element in level 0. From there, the reply will be forwarded along the lower output to primary input 3 as desired (note that in this level, only tag bit is considered in selecting the proper output of a combining element because the corresponding indicator tag bit is absent and can be viewed as "0" always). Our particular fault-tolerant structure makes it possible to have a simple process for establishing return paths correctly.

This design incurs a little extra cost yet improves the reliability of the overall combining architecture significantly. It can tolerate multiple failed combining elements, as long as at least one of an element and its conjugate is operational. To evaluate the degree of reliability improvement, let us assume that the failure rate of a combining element is exponential, with λ of (and the demultiplexer has a negligible failure rate). The reliability of our basic design with size N is $R_b(t) = e^{-(N-1)\lambda t}$, whereas the reliability of our fault-tolerant design is $R_f(t) = [1 - (1 - e^{-\lambda t})^2]^{N/2}$. At the time of λt equal to 10^{-3} , for example, a system with size 256 has reliability values of 0.7749 and 0.99974 respectively for the basic and fault-tolerant designs. This indicates that the fault-tolerant design gives rise to a reliability improvement factor (RIF, which is the ratio of $1 - R_b(t)$ to $1 - R_f(t)$) of as large as 865 for $\lambda t = 10^{-3}$.

4. CONCLUSIONS

An alternative combining architecture has been presented that can eliminate adverse hot-spot contention effectively at reduced hardware complexity, provided the number of concurrent hot-spot number is reasonably small (say, no more than 8 in a system). This combining architecture uses a combining tree to perform combining and a demultiplexer tree to deliver combined requests to their proper destinations. The reliability of the overall architecture can be improved significantly by making the combining tree fault-tolerant at a little additional cost. This architecture is applicable to large-scale multiprocessors for situations where hot-spot traffic is differentiable from regular traffic so that it is possible to direct hot-spot requests, after generated, to the combining architecture for combining.

REFERENCES

- [1] G. F. Pfister and V. A. Norton, "Hot-Spot Contention and Combining in Multistage Interconnection Networks," *IEEE Trans. Computers*, vol. C-34, pp. 943-948, Oct. 1985.
- [2] A. Gottlieb et al., "The NYU Ultracomputer - Designing an MIMD Shared Memory Parallel Computer," *IEEE Trans. Computers*, vol. C-33, pp. 175-189, Feb. 1983.
- [3] N.-F. Tzeng, "Design of a Novel Combining Structure for Shared-Memory Multiprocessors," *Proc. 1989 Int'l Conf. Parallel Processing*, Aug. 1989, pp. 1-8.
- [4] R. Lee, "On Hot Spot Contention," *ACM SIG Computer Architecture News*, vol. 13, pp. 15-20, Dec. 1985.
- [5] B. O. A. Grey, A. Avizienis, and D. A. Rennels, "A Fault-Tolerant Architecture for Network Storage Systems," *Proc. 14th Int'l Symp. Fault-Tolerant Computing*, June 1984, pp. 232-239.